Node.js — Day 26

Backend Organization, Validation & Business Logic

■ Today’s goal

Learn how to keep a Node.js backend clean by separating validation, business logic, and application coordination into focused modules.

Layer	Responsibility
app.js	**Coordinates the flow: get input → validate → call business logic → output result.**
validation.js	**Checks whether required input is valid.**
user.js	**Contains user-related business rules and creates the user object.**

1. Separating Validation

Validation answers: **“Is this input acceptable?”** Keeping validation in its own module prevents app.js from becoming noisy as the backend grows.

validation.js

```
function validateUser(name, age) {
    if (!name || !age) {
        return false;
    }
    return true;
}

module.exports = validateUser;
```

In app.js:

```
const validateUser = require("./validation");

if (!validateUser(name, age)) {
    console.log("Please provide name and age");
    process.exit(1);
}
```

Key idea: app.js uses validation without needing to know its internal implementation.

2. Business Logic

Business logic is the actual rule or decision of an application. Example: **cart total above ■1000 → 10% discount.**

Our user-related rules:

```
isAdult  → age >= 18
isSenior → age >= 60
```

user.js

```
function createUser(name, age) {
    const isAdult = age >= 18;
    const isSenior = age >= 60;

    return {
        name: name,
        age: age,
        isAdult: isAdult,
        isSenior: isSenior
    };
}

module.exports = createUser;
```

user.js owns the user-related rules; **app.js** receives the result.

3. app.js as the Coordinator

```
Input
  ↓
Validation
  ↓
Business Logic
  ↓
Output
```

Think of app.js like a project manager: it coordinates the work while specialized modules handle their own responsibilities.

4. Complete Day 26 Example

app.js

```
const validateUser = require("./validation");
const getUser = require("./user");

const name = process.argv[2];
const age = Number(process.argv[3]);

if (!validateUser(name, age)) {
    console.log("Please provide name and age");
    process.exit(1);
}

const user = getUser(name, age);
console.log("Name:", user.name);
console.log("Age:", user.age);
console.log("Adult:", user.isAdult);
console.log("Senior:", user.isSenior);
```

5. Practical Tests

```
node app.js Bhavana 26
Name: Bhavana
Age: 26
Adult: true
Senior: false

node app.js Ravi 65
Name: Ravi
Age: 65
Adult: true
Senior: true
```

6. Why This Structure Matters

As an application grows, keeping validation, business rules, input handling, and output in one file makes code harder to understand and maintain.

Rule: one file → one main responsibility. This does not mean one function per file; related responsibilities can stay together.

7. Day 26 Key Takeaways

• **Validation** checks whether input is acceptable.
• **Business logic** contains application/domain rules and decisions.
• **app.js** mainly coordinates the application flow.
• Separate modules make code cleaner and easier to maintain.
• **require()** loads a module; **module.exports** exposes functionality.

■ Mental model

app.js = Coordinator
validation.js = Gatekeeper
user.js = Business Rules / User Logic

8. Final Assessment

You answered all three assessment questions correctly: **3/3** ■

Q1 — app.js coordinates the flow between modules.
Q2 — The “age ≥ 18 means adult” rule belongs in user.js.
Q3 — Separation of responsibilities keeps the backend easier to maintain.

■ Day 26 Status: COMPLETE

You implemented the concepts, tested multiple inputs, verified the output, and passed the final assessment.